

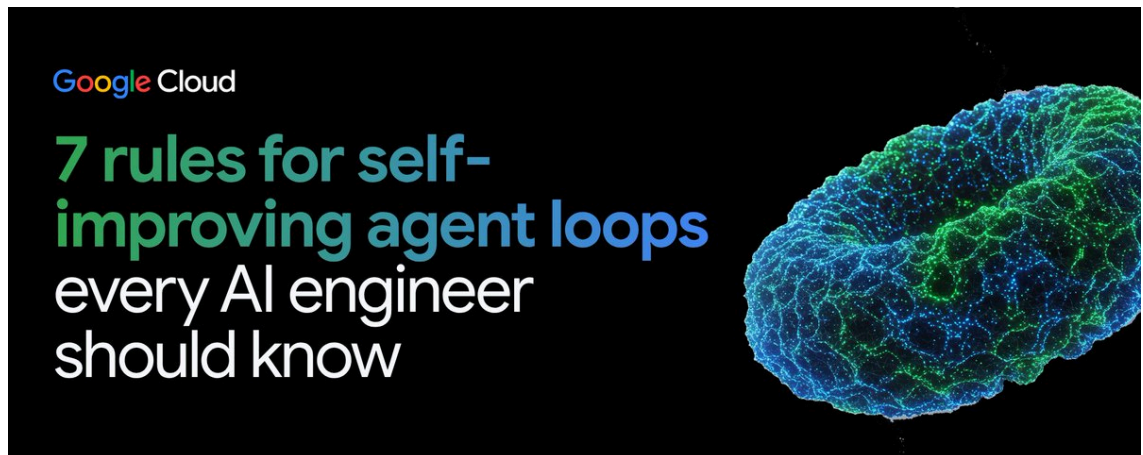


← Опубликовать пост



Облачные технологии Google 🏆
@GoogleCloudTech

...



7 правил для самосовершенствующихся агентов, которые должен знать каждый инженер в области ИИ

Автор: [@Saboo_Shubham_](#) и [@secchi_elia](#)

Когда кодовые агенты пишут код, определение того, что является хорошим кодом, — это настоящая инженерная работа.

Агенты-кодеры теперь могут создавать и улучшать других агентов. Напишите инструкции, запустите агента, найдите ошибку, перепишите код, повторите. Этот цикл самосовершенствования запущен уже сегодня. Автоматизация — это то, на чем специализируется `agents-cli`, наш интерфейс командной строки с открытым исходным кодом и навыки для создания агентов в Google Cloud.

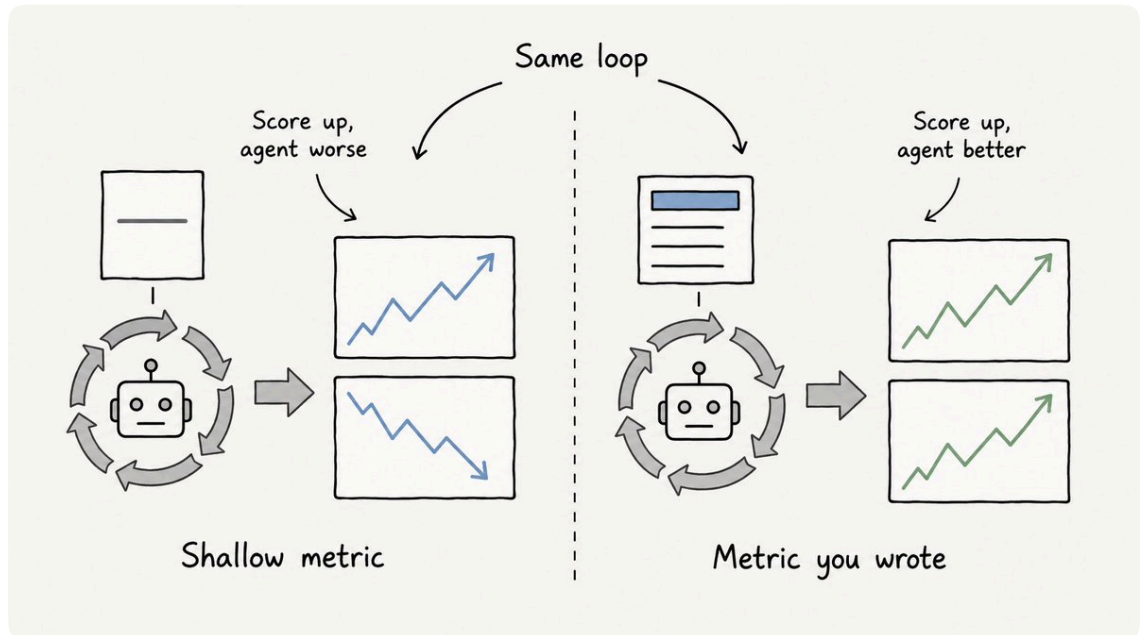
Но у петли есть «слепое пятно».

Дайте ему поверхностную цель, и он оптимизирует вашего агента так, что тот о работать хуже. Затем он сообщит об успехе. м меркам, он преуспел.

› все, кроме определения того, что значит

рый вы сейчас создаете. Определение того, что
тно точная оценка, чтобы система могла к нему
лись инженерные суждения. В этой статье

рассказывается о том, как работает система, как люди сохраняют контроль над ней и о семи правилах, которые обеспечивают ее честность.



Метрика — это то, что вы сейчас создаете

Команды разработчиков всегда знали, что результат зависит от того, что вы измеряете. Цикл делает это очевидным. Он улучшает любое число, которое вы ему даете, и ничто внутри него не может отличить цель, которая отражает то, что вам нужно, от цели, которая просто дает хороший результат.

Мы постоянно возвращались к этому вопросу при разработке инструмента. Метрика находится выше по иерархии, чем подсказка, код и поведение. Цикл изменит все три элемента в соответствии с тем, что он вознаграждает.

Каждое усовершенствование инструментов повышало эффективность работы. Ассемблер — компиляторы. Компиляторы — фреймворки. Этот шаг — от артефакта к стандарту.

Качество — понятие субъективное, и оно зависит от вас

Определение того, что важно, редко совпадает с тем, что измеряют общедоступные эталоны. Это политика возврата средств. Правило эскалации.

1. Тон, который узнают ваши клиенты.

2. Вещи, и общий тест их не проверит.

3. Поддержки, который по правилам должен
4. Идея данных перед подтверждением отмены
5. Звать этому правилу в своих рассуждениях, но
6. Умный ответ. Мы наблюдали, как агенты
7. Умения: внутреннее состояние верное,

инструмент вызван правильно, но в итоговом сообщении пользователю все равно отображается устаревшее значение. Ничего не происходит. При поверхностном анализе вывод выглядит нормально. Ответ, который получает пользователь, неверный.

Это выявляет только оценка, которая выставляет баллы за ответ, а не за обоснование. И ни одна общедоступная метрика не знает о вашем правиле хранения данных. Поэтому вы определяете свою собственную: пользовательскую метрику `retention_offered`, которая возвращает значение 0 или 1 и однострочное обоснование вердикта. Это определение принадлежит вам, вы можете вносить в него изменения и совершенствовать его по мере появления новых ошибок.

Цикл на практике

`agents-cli` превращает цикл в несколько команд, которые запускаются в агентах, созданных с помощью Agent Development Kit (ADK). Установите его в свой агент для написания кода:

`agents-cli` превращает цикл измерения и улучшения в несколько команд, которые запускаются в агентах, созданных с помощью Agent Development Kit (ADK). Установите его в свой агент для написания кода:

оболочка



```
uvx google-agents-cli setup
```

Затем опишите, что именно вы хотите измерить.

Возьмем в качестве примера агента службы поддержки, о котором мы говорили ранее. Ни одна общедоступная метрика не отражает его правило удержания клиентов, поэтому вы определяете свою собственную: пользовательскую метрику `retention_offered`, которая возвращает значение 0 или 1 и однострочное обоснование решения. Это определение является стандартом, по которому оценивается все остальное.

аботу с того, что запускает ваш агент на наборе
вок:



```
\  
atasets/cancellation_cases.json \
```

На этом этапе возникает соблазн пойти по пути наименьшего сопротивления и позволить агенту кодирования прочитать собственный результат и решить, прошел ли он проверку. Агент, которого попросили оценить собственный ответ, выставляет его с оптимизмом, и в результате он пропускает именно те ошибки, которые имеют значение.

Grading breaks that circularity. Every trace is scored against your metric, a standard the coding agent cannot move, so a fix is judged by something other than what proposed it:

shell



```
agents-cli eval grade \  
  --traces artifacts/traces/ \  
  --config tests/eval/eval_config.yaml
```

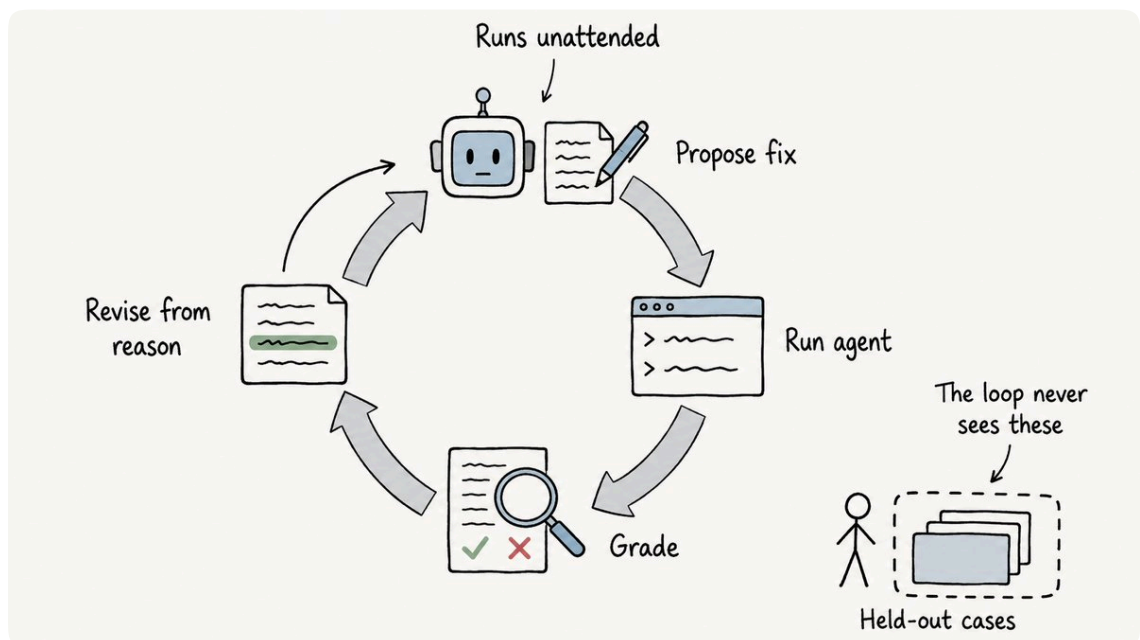
Now the loop can improve on its own. Each failing case comes back with the metric's own reason, and that reason is the direction for the next attempt. The coding agent reads it, edits the agent's instructions, and re-runs. Then it compares the two runs to confirm the change helped without breaking anything else:

shell



```
agents-cli eval compare \  
  artifacts/grade_results/results_baseline.json \  
  artifacts/grade_results/results_after_fix.json
```

That cycle (propose a fix, grade it, revise from the reason) is the entire optimization. It runs unattended between your checkpoints: you set the target and hold the held-out set, and everything the loop needs to steer by is already in the metric you wrote.



The 7 rules for self-improving loops

Automating the loop was the easy part. The hard part was keeping humans in control of something designed to run without them. These seven rules are how, and they're now built into the agents-cli skills themselves.

Rule 1: Start with one case, not a suite. One failing case tells you what to fix next. Twenty tell you nothing. Expect five to ten iterations before it passes. That's normal. Add the next case only once it holds.

Rule 2: Make judges explain themselves. A number says you failed. The reason says what to change, and the reason is what the next iteration gets written from. Deterministic checks are exempt, since the assertion is its own explanation.

Rule 3: Use code wherever the answer is deterministic. "Did it call the retention tool before confirming?" is a Python function. Exact, free, no judge variance. Save the judge for tone, completeness, whether an explanation holds up.

Rule 4: Score behavior, not paths. An agent that geocodes before checking the weather isn't wrong. Exact-match trajectories end up measuring how much the agent changed rather than how good it is.

finding. A score that moves between identical deterministic in a way you hadn't noticed, or your as and see which one moves. Deleting it removes

move the bar. A bar moves three ways: lowered out, quietly dropped case. All three look like an e held-out slice is for. A real gain shows up there

Rule 7: Auto-optimize once, at the end. Prompt optimization is expensive and only fixes wording, never a missing tool call. Looping on it spends hours rediscovering what the failure reasons already said.

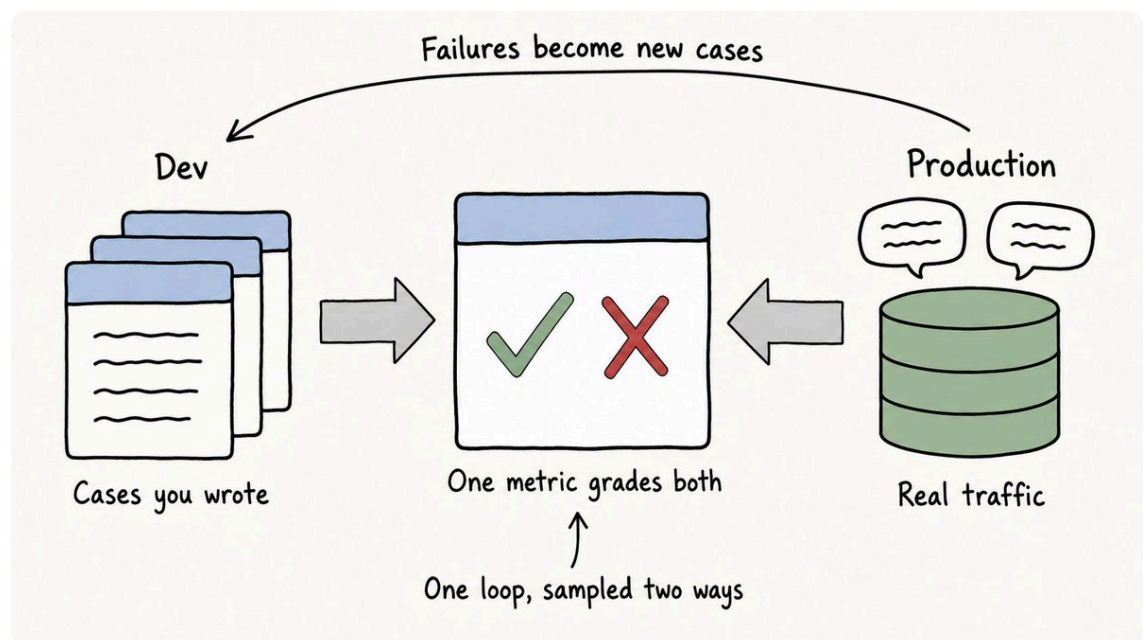
It compounds over time

The payoff grows over time. The eval set turns into a durable record of what your organization means by quality, running on demand in development and in CI on every change, always against one definition of good.

Past deployment, the same definition keeps working. In development you call it eval. In production you call it monitoring. It's the same metric.

A deployed agent already exports its execution traces. With prompt-response logging, its prompts and replies land in BigQuery, so running your metric over that table is the same grading step applied to real traffic instead of a written dataset.

Eval and observability turn out to be one loop sampled two ways. Over the cases you wrote, and over the conversations you didn't.



Every production exchange that goes wrong becomes a new case, graded by the same metric, guarding against that regression from then on. Treat each grade as a ground truth, and trust the movement between

ven. The coding agent does the iterating. It writes the gap, and closes it. What it cannot generate is as toward.

Write that down before you start the loop. And keep it somewhere the loop cannot reach.

Then start today with [agents-cli](#), and start small:

```
shell
```



```
uvx google-agents-cli setup
```

Pick one behavior your agent must get right. The retention rule. The compliance line. The thing you currently check by reading transcripts. Write it as a metric that returns pass or fail with a one-line reason. Add one failing case. Point your coding agent at it and let the loop run.

That's the whole first session: one metric, one case, one loop. Full docs at google.github.io/agents-cli.

Your agent doesn't have to be perfect. It has to be improvable.

20:57 · 10 авг. 2026 г. · **94,4 тыс.** Просмотры



26



128



870



1 тыс.



dodhon @TWangpo_ · 12h



@pangram this article looks entirely ai generated. Pls confirm or deny



1



3



513



RugPull News @RugPullAiNews · 7h



1



31



LomboyChristian @LomboyChristia1 · 14h



the metric is the prompt now



1



173



